Chapter 5

Advanced Encryption Standard (AES)

Outlines

1. Introduction to Advanced Encryption Standard (AES).
2. Design of AES.
3. Comparison of AES with DES.
4. Security of AES.

Learning Objectives

In the end of this Chapter 5, students will be able:

1. To understand the birth of **Advanced Encryption Standard (AES)**.
2. To understand the **design** of AES, followed by its operations.
3. To understand and differentiate the **difference between AES with DES**.
4. To understand some **security aspects** of AES.

ADVANCED ENCRYPTION STANDARD (AES)

HISTORY

Advanced Encryption Standard (AES) – History

- A symmetric-key block cipher, AES was published by the **National Institute of Standards and Technology (NIST)** in December 2001.
- It was designed by Belgium academics J. **Dae**men and V. **Rij**men, hence AES is also known as **Rijndael** algorithm.
- The selection of AES falls into the three criteria:
  1. **Security**: Minimum key size to resist cryptanalysis.
  2. **Cost**: Computational efficiency and storage requirement for various implementation.
  3. **Implementation**: Flexibility and simplicity in various platforms.

Advanced Encryption Standard (AES) – History

- AES operates on **128-bit plaintext block** with **either 128-bit, 192-bit, or 256-bit key**. The key size used can be understood immediately from the version's names.
  - **Example**: **AES-128** indicate the key size used is 128 bits. Another two versions are **AES-192** and **AES-256**.
  - Regardless of the key-size choice, the **plaintext** is always fixed at **128-bit block**.
- It is an **iterative** cipher and **NOT Feistel** cipher. (We will discuss this issue in the later slides)

ADVANCED ENCRYPTION STANDARD (AES)

DESIGN

AES – Overview Design

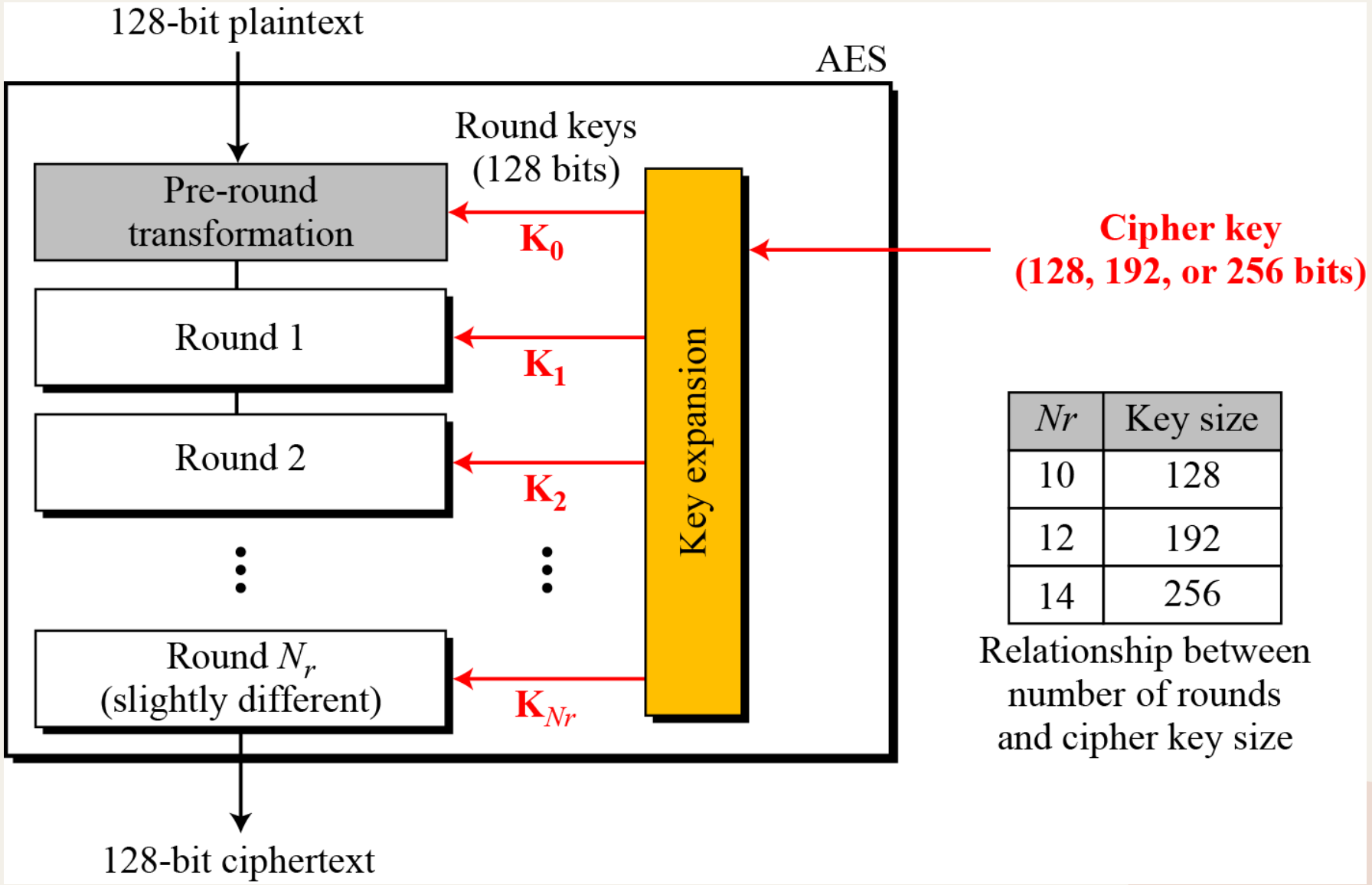
- AES is a **Non-Feistel cipher** that encrypts and decrypts a plaintext block of 128 bits.
- It comes with **three versions** based on the choice of **key-size**:
  1. AES-128: Uses **128-bit** key, with **10 rounds** operation.
  2. AES-192: Uses **192-bit** key, with **12 rounds** operation.
  3. AES-256: Uses **256-bit** key, with **14 rounds** operation.
- Regardless of the version, both **plaintext block**, and **round keys** (used in single round operation) is always fixed at **128 bits**.

Notice that the number of round keys generated is always **ONE MORE** than the number of rounds.

This is due to the **Pre-round Transformation** that requires one round key in its operation.

Mathematically,

$$\textbf{No. of rounds} = N_r + 1$$



Nr	Key size
10	128
12	192
14	256

Relationship between number of rounds and cipher key size

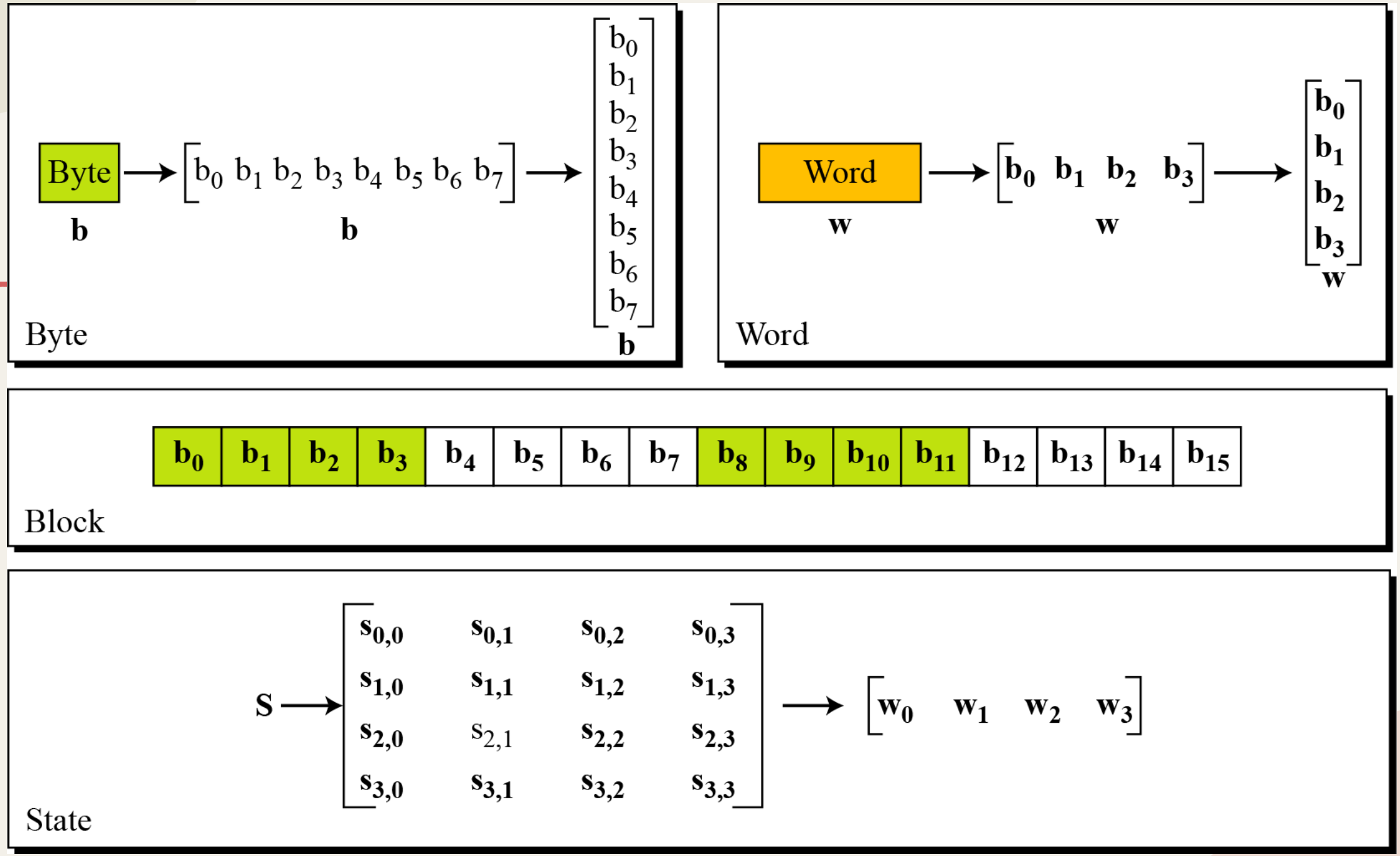
AES – Data Units

- AES uses **five units** of measurement to refer to data:

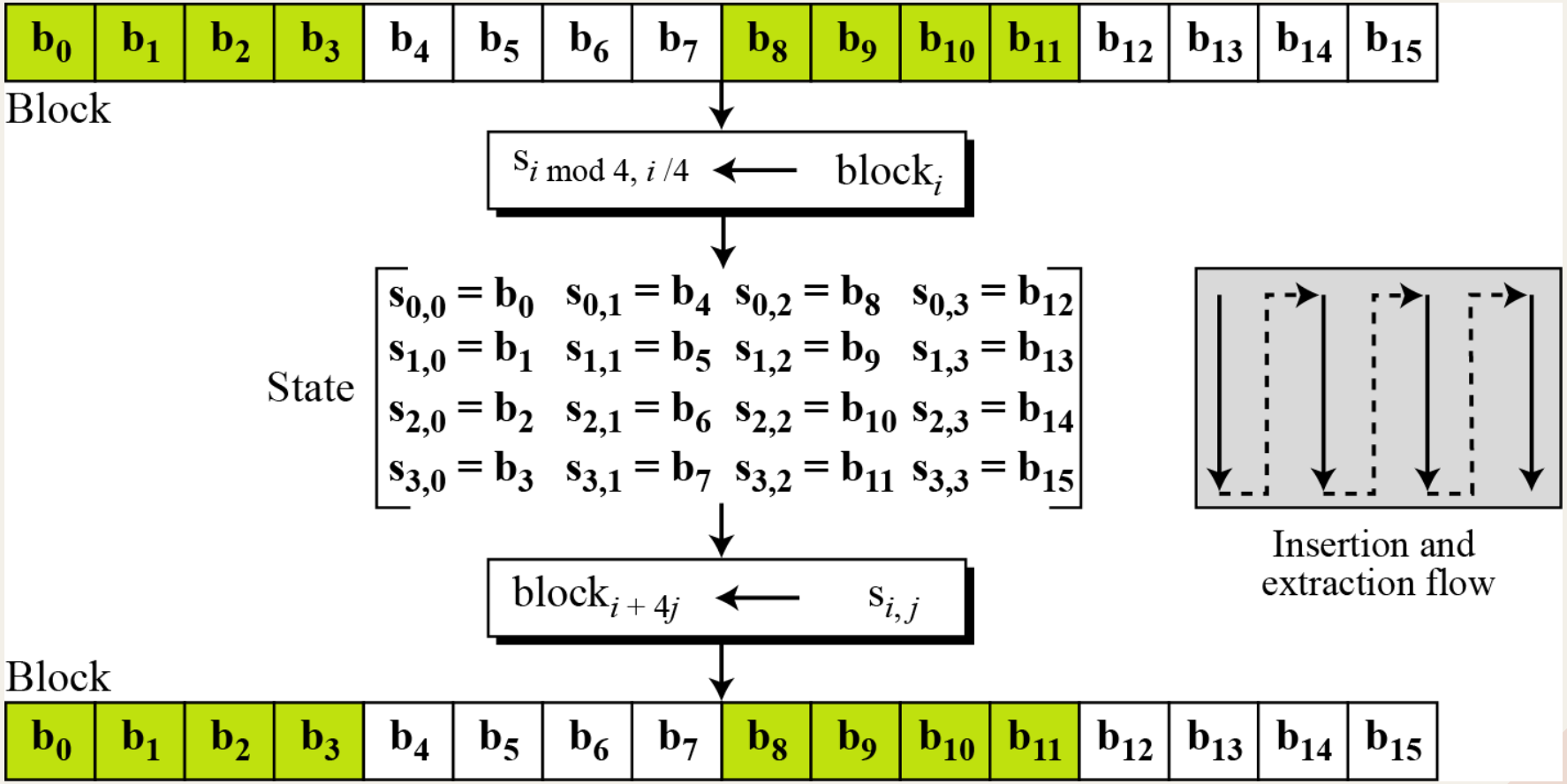
1. **Bit**: A binary digit with a value of 0 or 1.
2. **Byte**: A group of 8 bits, treated as a row matrix (1×8) of eight bits, or a column matrix (8×1) of 8 bits.
3. **Word**: A group of 32 bits, treated as a row matrix of 4 bytes, or a column matrix of 4 bytes.
4. **Block**: A group of 128 bits, represented as a row matrix of 16 bytes.

AES – Data Units

5. **State**:
   - AES uses several **rounds** in which each round is made up of several **stages**. The data block is transformed from one stage to another.
   - At the beginning and end of the cipher, AES uses the term **data block**.
   - Before and after each stage, the data block is referred to as a **state**.
   - A group of 128 bits, treated as matrices of 4×4 bytes or as a row matrix (1×4) of words.


Byte
b
b0 b1 b2 b3 b4 b5 b6 b7
b
b0
b1
b2
b3
b4
b5
b6
b7
b
Byte
Word
w
b0 b1 b2 b3
w
b0
b1
b2
b3
w
Word
b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 b10 b11 b12 b13 b14 b15
Block
S
s0,0 s0,1 s0,2 s0,3
s1,0 s1,1 s1,2 s1,3
s2,0 s2,1 s2,2 s2,3
s3,0 s3,1 s3,2 s3,3
w0 w1 w2 w3
State

AES – Data Transformation



AES – Data Transformation

- The operations in AES involves the transformation of data **block** (plaintext) in the following procedures:

1. Each plaintext letter is converted into **hexadecimals (base 16)**.
2. These hexadecimals is then arranged into **column vectors** to form a **state** (**4 bytes** per column).
3. The state undergoes AES operation (we will go through this in detail later).
4. The output is transformed back into **block** before the next operation.

AES – Data Transformation

- Example of block to state transformation of a plaintext. If the block is **short of** data (not a multiples of 16), a padding using some arbitrary letters (such as X and Z) is done.

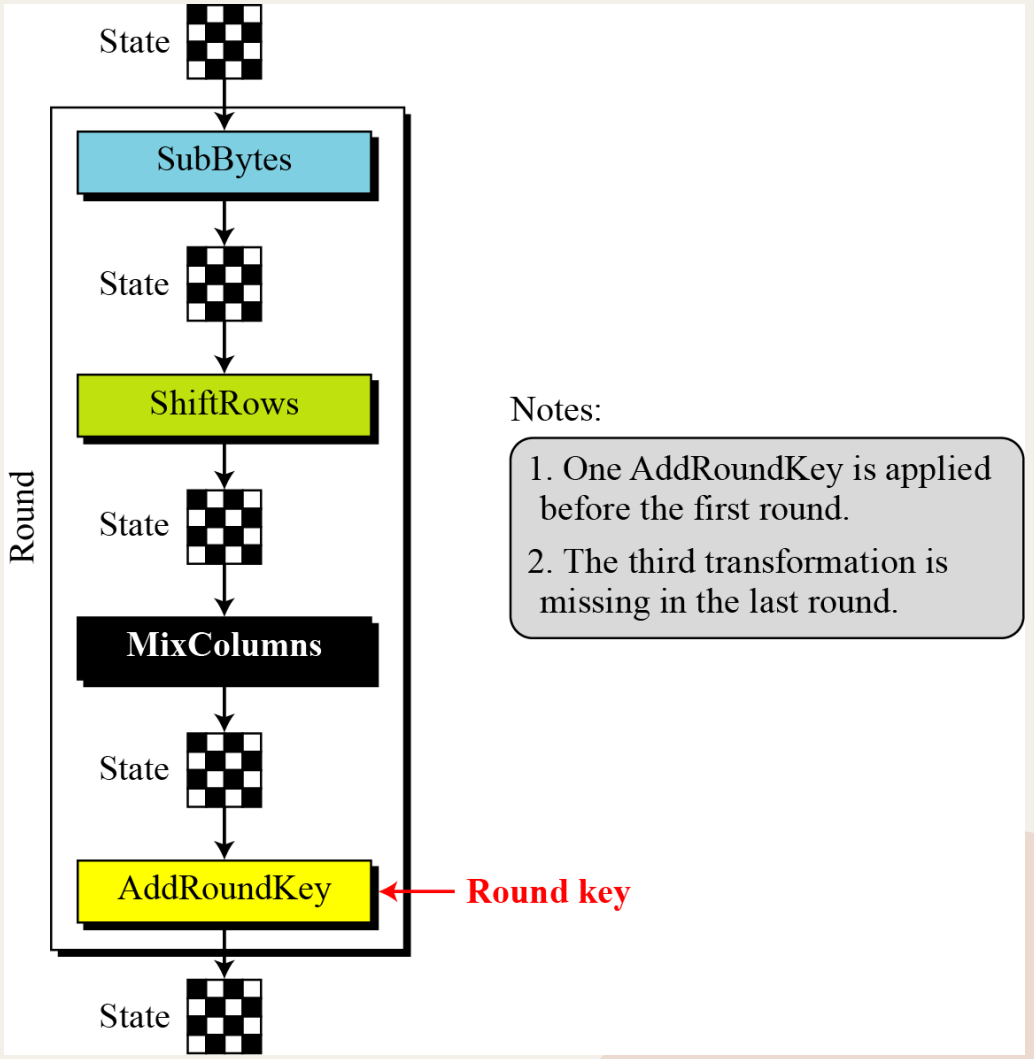
Text	A	E	S	U	S	E	S	A	M	A	T	R	I	X	Z	Z
Hexadecimal	00	04	12	14	12	04	12	00	0C	00	13	11	08	23	19	19

$$\begin{bmatrix} 00 & 12 & 0C & 08 \\ 04 & 04 & 00 & 23 \\ 12 & 12 & 13 & 19 \\ 14 & 00 & 11 & 19 \end{bmatrix}$$ State

AES – Round Operations (Overview)

- The right figure shows the **complete one-round** of AES encryption. Each round contains 4 transformation of:

1. ***SubBytes***
2. ***ShiftRows***
3. ***MixColumns*** (The **MOST complicated** operation in a round)
4. ***AddRoundKey***



AES – Round Operations (Overview)

- Each transformation takes a state and creates another state to be used for the next transformation or the next round.

➢ **Encryption**:

✓ **Pre-round** performs only ***AddRoundKey*** transformation.

✓ **Last round** do **NOT** undergo ***MixColumns*** transformation.

➢ **Decryption**:

✓ Uses inverse transformations of all the operations. Each transformation is either invertible via some fixed rules or self-invertible.

AES – Round Operations (Overview)

- In summary, for a specific AES version, the total number of rounds and round keys required for each transformation are summarized in the following table.

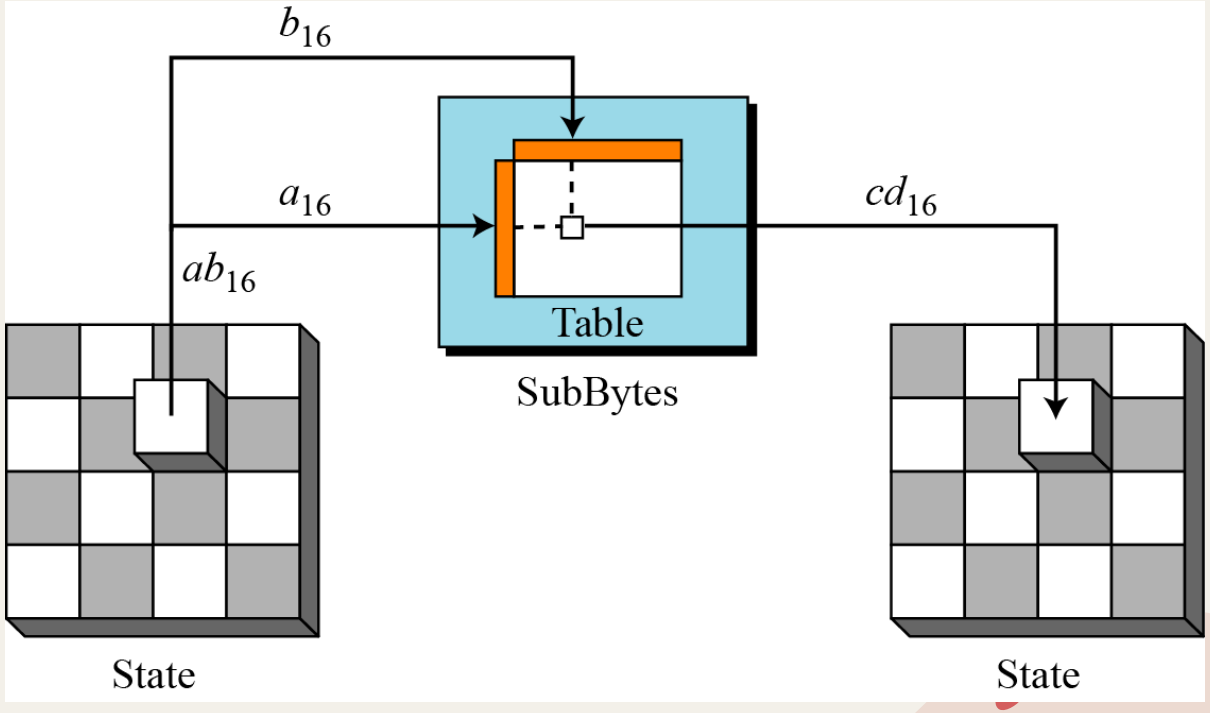
Version	Block Size	Key Size	Round Key Size	Number of Rounds	Number of Round Keys	Number of Transformation
AES-128	128	128	128	10	11	$1 + 9(4) + 3 = 40$
AES-192	128	192	128	12	13	$1 + 11(4) + 3 = 48$
AES-256	128	256	128	14	15	$1 + 13(4) + 3 = 56$

ADVANCED ENCRYPTION STANDARD (AES)

SubBytes Substitution

AES – SubBytes Substitution

- The SubBytes operation involves 16 **independent byte-to-byte** transformation (**substitution**).
- The bytes is transformed into hexadecimals, with the **1st digit** corresponds to the **ROW**, and the **2nd digit** corresponds to the **COLUMN** of the ***SubBytes*** Table.
- The **S-Box** and its **Inverse S-Box** are given in next slides.



x \ y	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

(a) S-box

Input
3A F
Outpu
80 A1

Exerci
85 =>
53 =>
CC =>

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
	1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
	2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
	3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
	4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
	5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
	6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
	7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
	8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
	9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
	A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
	B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
	C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
	D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
	E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
	F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

(b) Inverse S-box

$3A\ F1$

80 A

AES – SubBytes Substitution

❖ **Example 1**: Perform the following substitutions for the hexadecimals.

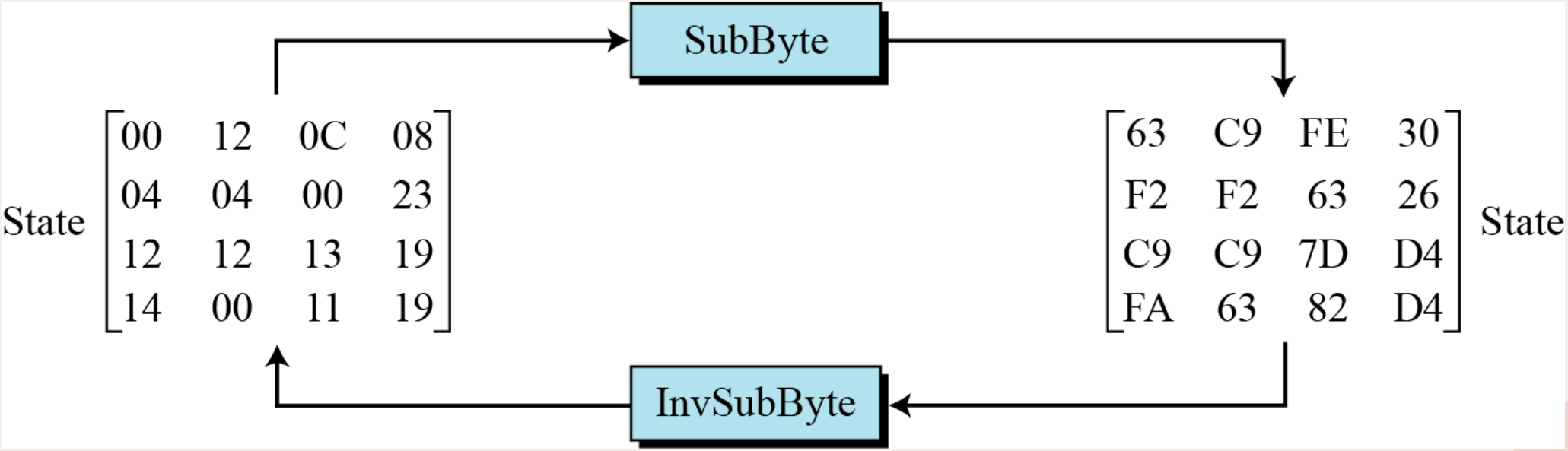
1. $3A\ \ F1\ \ B2\ \ 99$ via **S-Box** Substitution
2. $3A\ \ F1\ \ B2\ \ 99$ via **Inverse S-Box** Substitution

❖ **Solutions**:

1. $80\ \ A1\ \ 37\ \ EE$
2. $A2\ \ 2B\ \ 3E\ \ F9$

AES – SubBytes Substitution

- **Example 2**: The figure shows a demonstration of ***SubBytes*** and ***Inverse SubBytes*** on a full state.

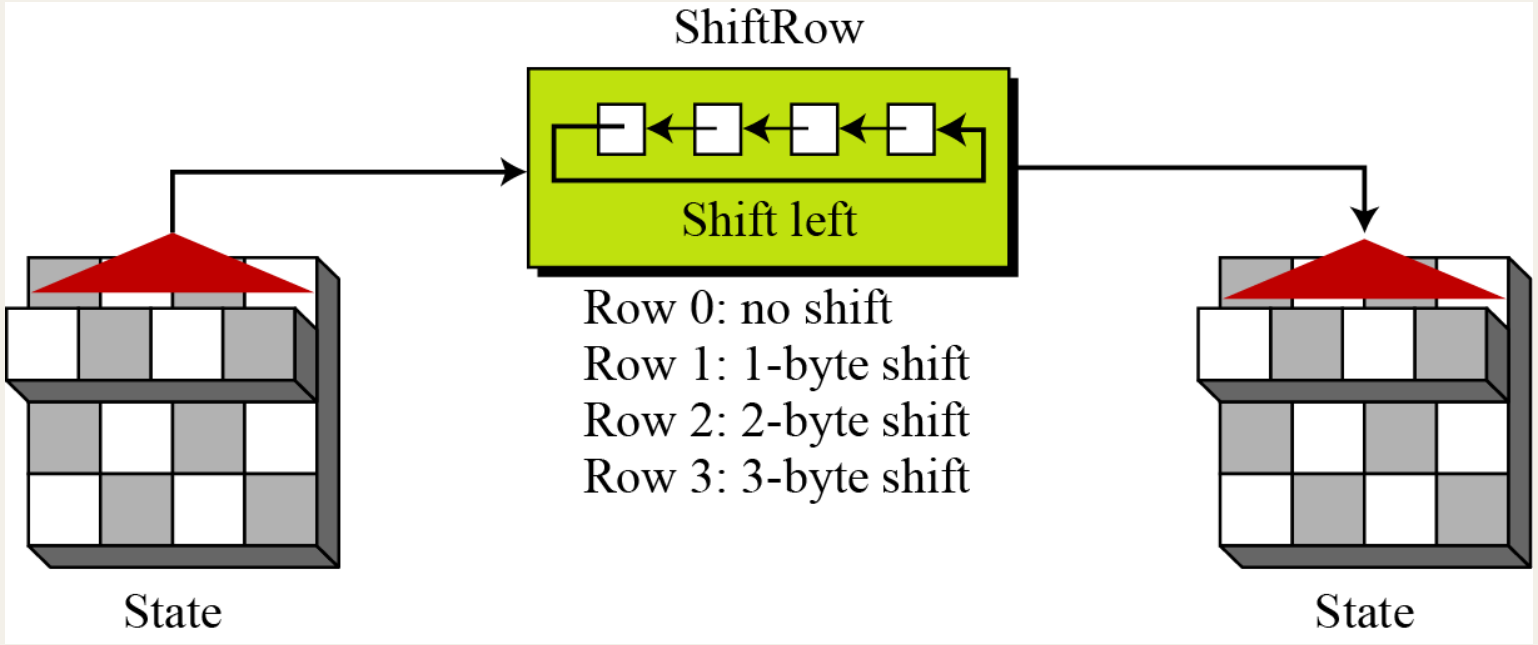


ADVANCED ENCRYPTION STANDARD (AES)

ShiftRows

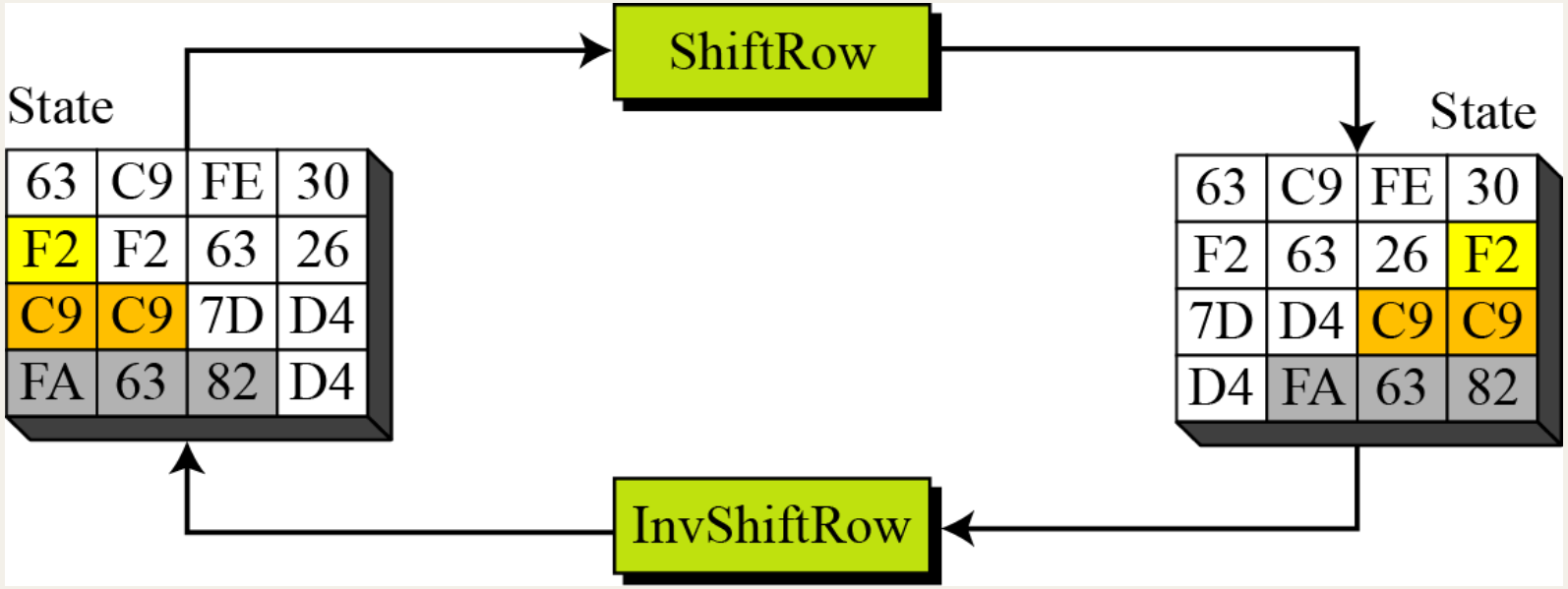
AES – ShiftRows

- After the ***SubBytes*** transformation, the state undergoes the ***ShiftRows*** operations, in which each row is **permuted** (shifted different bytes to the **LEFT**) as shown below.



AES – ShiftRows

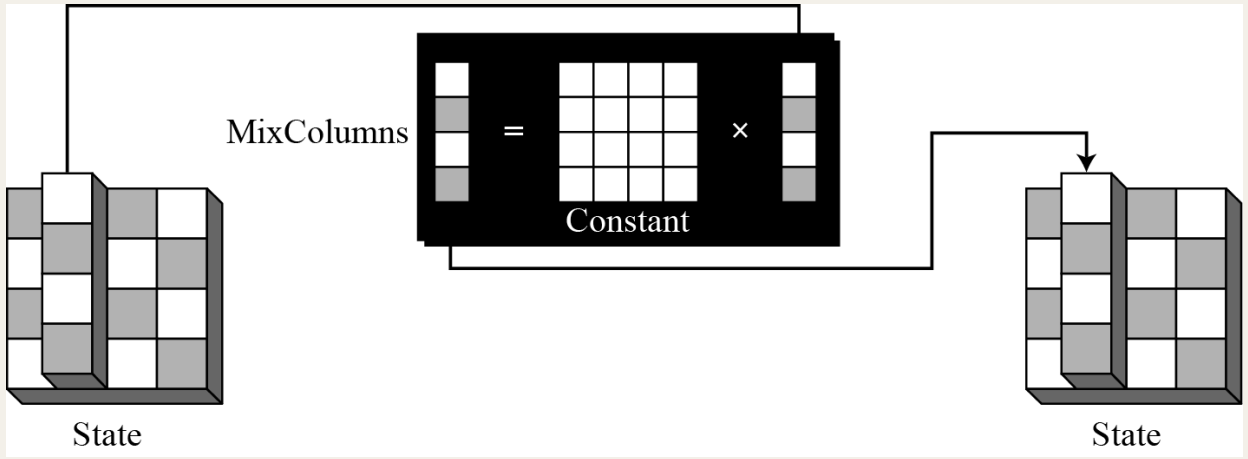
- **Example 3**: We consider the following state and its operation via ***ShiftRows***. Note that the ***Inverse ShiftRows*** is just RIGHT-shift the same bytes of ***ShiftRows***.



ADVANCED ENCRYPTION STANDARD (AES)

MixColumns

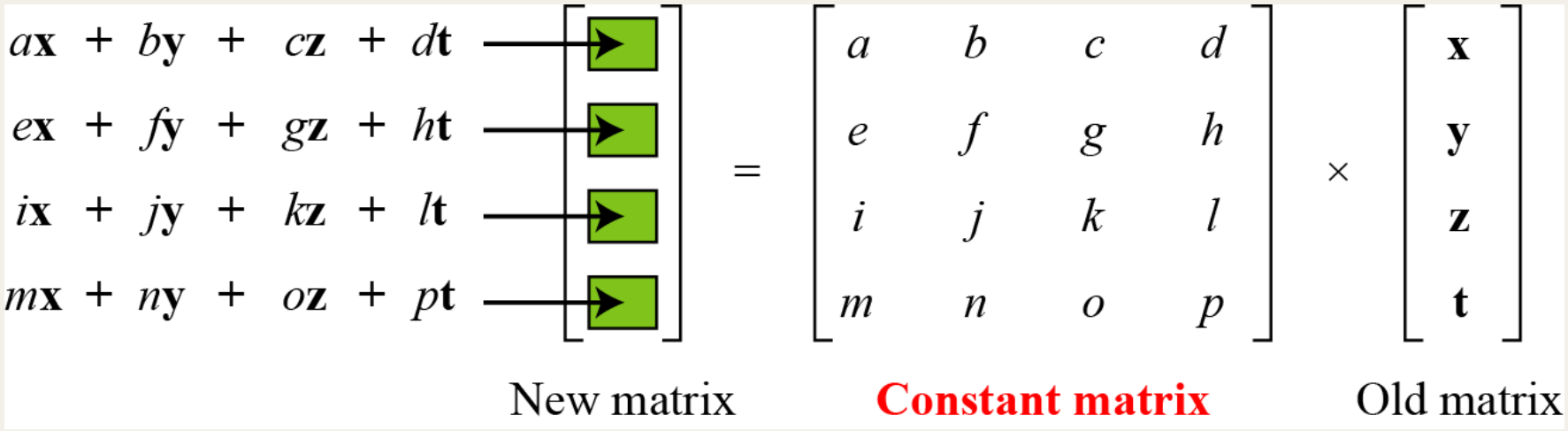
AES – MixColumns



- ***MixColumns*** probably the most complicated part in AES, since there is no fixed table to refer to (like ***SubBytes***).
- It operates at the **column level**, by transforming each column of the state to a new column in another state.
- The idea of ***MixColumns*** is to change the bits inside a byte, based on the bits inside the neighboring bytes (**inter-byte** transformation) using **matrix multiplication** with a **key matrix**.

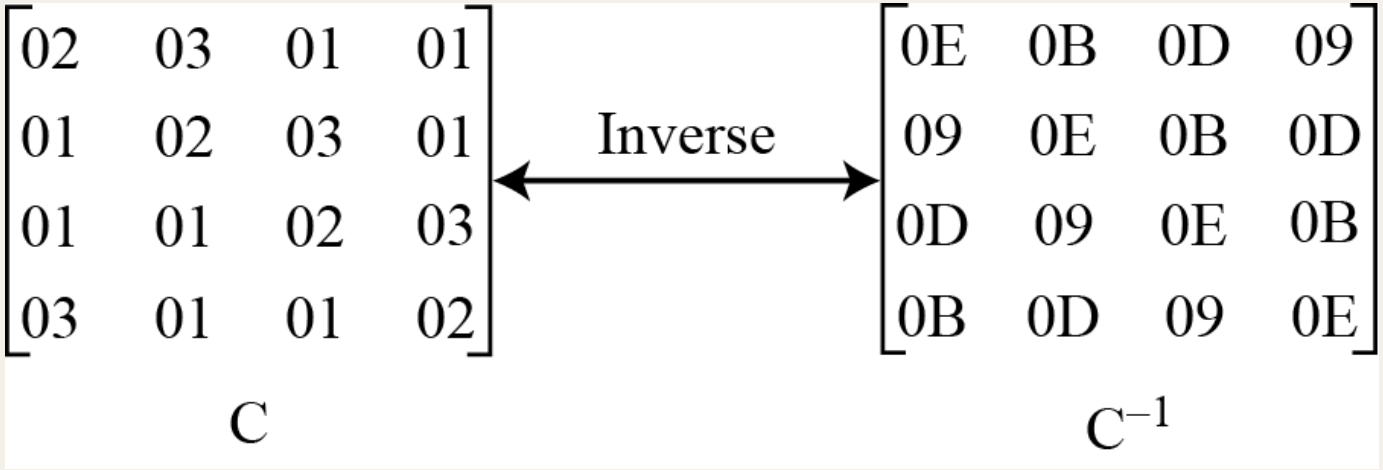
AES – MixColumns

- The following illustrates the bytes-mixing using matrix multiplication.



AES – MixColumns

- In AES, the key matrix (constant matrix) and its inverse are fixed as follows.



AES – MixColumns

- The multiplication of ***MixColumns*** is **NOT** simply multiplying and adding each row to column as in normal matrices. Try the following example and observe it yourself.

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \times \begin{bmatrix} 63 & C9 & FE & 30 \\ F2 & 63 & 26 & F2 \\ 7D & D4 & C9 & C9 \\ D4 & FA & 63 & 82 \end{bmatrix} = \begin{bmatrix} 62 & 02 & 27 & 26 \\ CF & 92 & 91 & 0D \\ 0C & 0C & F4 & D6 \\ 99 & 18 & 30 & 74 \end{bmatrix}$$

Example. How 62 is obtained. + = XOR

(02 x 63) XOR (03 x f2) XOR (1 x 7d) XOR (1 x d4)

1. 02 = 0000 0010 = X ; 63 = 0110 0011 = $x^6 + x^5 + x + 1$

02 * 63 = $x^7 + x^6 + x^2 + x = 1100\ 0110$

2. 03 = 0000 0011 = (X + 1); F2 = 1111 0010 = $x^7 + x^6 + x^5 + x^4 + x$.

03xf2 = $(x + 1)(x^7 + x^6 + x^5 + x^4 + x)$

$= (x^8 + \cancel{x^7 + x^6 + x^5} + x^2) + (\cancel{x^7 + x^6 + x^5} + x^4 + x)$

$= x^8 + x^4 + x^2 + x = (\cancel{x^4} + x^3 + \cancel{x} + 1) + \cancel{x^4} + x^2 + \cancel{x} =$

$= x^3 + x^2 + 1 = 0000\ 1101$

7d = 0111 1101

D4 = 1101 0100

02x63	1	1	0	0	0	1	1	0
03xf2	0	0	0	0	1	1	0	1
7d	0	1	1	1	1	1	0	1
d4	1	1	0	1	0	1	0	0
62 (xor)	0	1	1	0	0	0	1	0

Inverse mix column

$$\begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \times \begin{bmatrix} 62 & 02 & 27 & 26 \\ CF & 92 & 91 & 0D \\ 0C & 0C & F4 & D6 \\ 99 & 18 & 30 & 74 \end{bmatrix} = \begin{bmatrix} 63 & C9 & FE & 30 \\ F2 & 63 & 26 & F2 \\ 7D & D4 & C9 & C9 \\ D4 & FA & 63 & 82 \end{bmatrix}$$

(0e * 62) XOR (0b*CF) XOR (0D * 0c) XOR (09 * 99) = 63

1. 0e = 0000 1110 = $x^3 + x^2 + x$; 62 = 0110 0010 = $x^6 + x^5 + x$

0e x 62 = $(x^3+x^2 + x)(x^6 + x^5 + x) = (x^9 + \cancel{x^8}+x^4) + (\cancel{x^8}+\cancel{x^7} + x^3) + (\cancel{x^7}+x^6 + x^2)$

$= x(x^4 + x^3 + x + 1) + x^6 + x^4 + x^3 + x^2 = (\cancel{x^4+x^2} + x) + x^6 + x^5 + \cancel{x^4} + x^3 + \cancel{x^2} = x^6 + x^5 + x^3 + x$

= 0110 1010 => 6A

2. (0b * cf)

0b = 0000 1011 = $x^3 + x + 1$; cf = 1100 1111 = $x^7 + x^6 + x^3 + x^2 + x + 1$

(0b *cf) = $(x^3+x + 1)(x^7 + x^6 + x^3 + x^2 + x + 1) = x^{10} + x^9 + x^8 + [x^5+x^4 + x^3 + 1]$

$= x^2(x^4 + x^3 + x + 1) + x\ (x^4 + x^3 + x + 1) + (x^4 + x^3 + x + 1) + [x^5+x^4 + x^3 + 1]$

$= x^6 + x^5 + x^3 \Rightarrow$ 0110 1000 = 0x68

Inverse mix column

1011 0101

3. (0d * 0c)

0d = 0000 1101 ; $x^3 + x^2 + 1$; 0c = 0000 1100 = $x^3 + x^2$

(0d * 0c) = $(x^3+x^2 + 1)(x^3 + x^2) = (x^6 + \cancel{x^5} + x^3) + (\cancel{x^5} + x^4 + x^2)$ = 0101 1100

1111 1011

4. (09 * 99)

09 = 0000 1001 =$x^3 + 1$; 99 = 1001 1001 =$x^7 + x^4 + x^3 + 1$

(09*99) = $(x^3 + 1)(x^7 + x^4 + x^3 + 1) = x^{10} + \cancel{x^7} + x^6 + \cancel{x^3} + \cancel{x^7} + x^4 + \cancel{x^3} + 1$

$= x^2(x^4 + x^3 + x + 1) + x^6 + x^4 + 1 = \cancel{x^6} + x^5 + x^3 + x^2 + \cancel{x^6} + x^4 + 1$ = 0011 1101

0e*62	0	1	1	0	1	0	1	0
0b*cf	0	1	1	0	1	0	0	0
0d*0c	0	1	0	1	1	1	0	0
09*99	0	0	1	1	1	1	0	1
0x63 (x0r)	0	1	1	0	0	0	1	1

AES – MixColumns

- https://youtu.be/WPz4Kzz6vk4?t=613
- The multiplication, instead works under ***Galois Field*** $GF(2^8)$ (finite field) **reduction using irreducible polynomial**, that is $\mathrm{mod}\ p(x)$ where:

$$p(x) = x^8 + x^4 + x^3 + x + 1$$

$$x^8 = x^4 + x^3 + x + 1$$

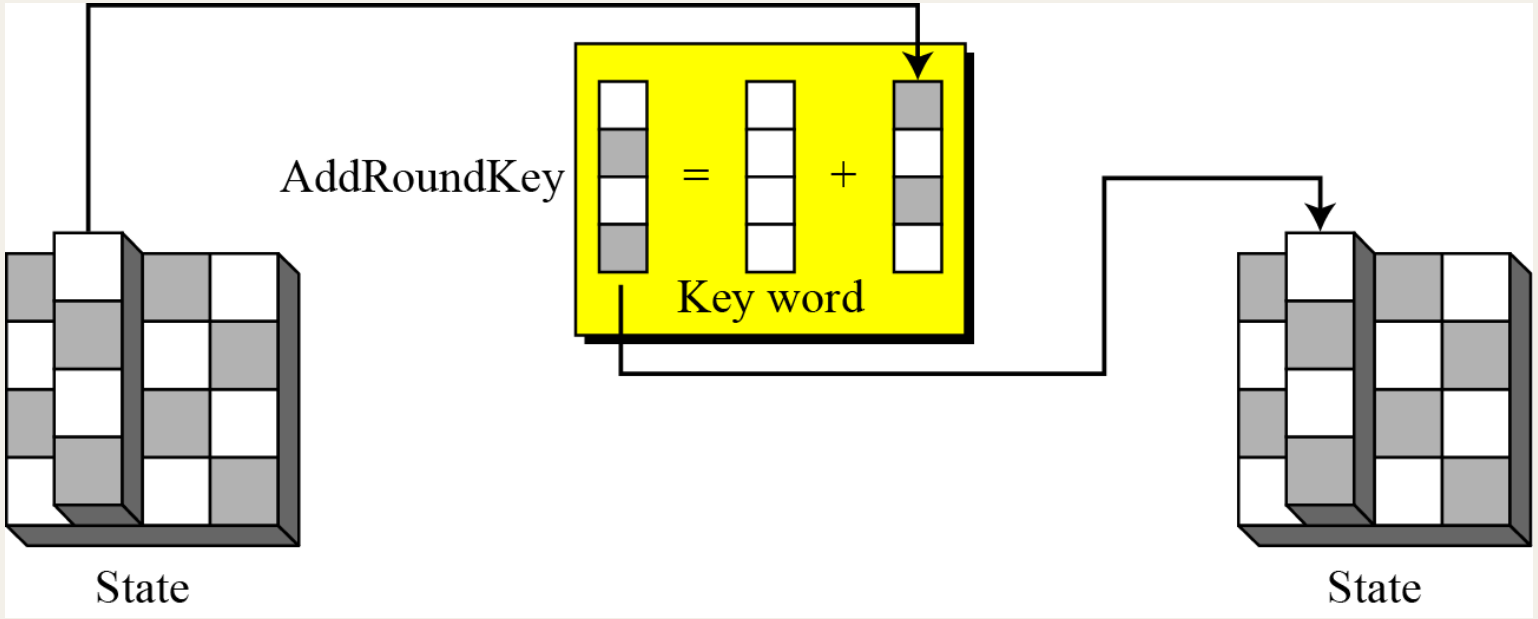
- **Note**: We will through the calculation in the lecture class, but we will do it in the tutorial sessions. The idea is similar to modular arithmetic we learned previously, just we mod a polynomial instead of an integer.

ADVANCED ENCRYPTION STANDARD (AES)

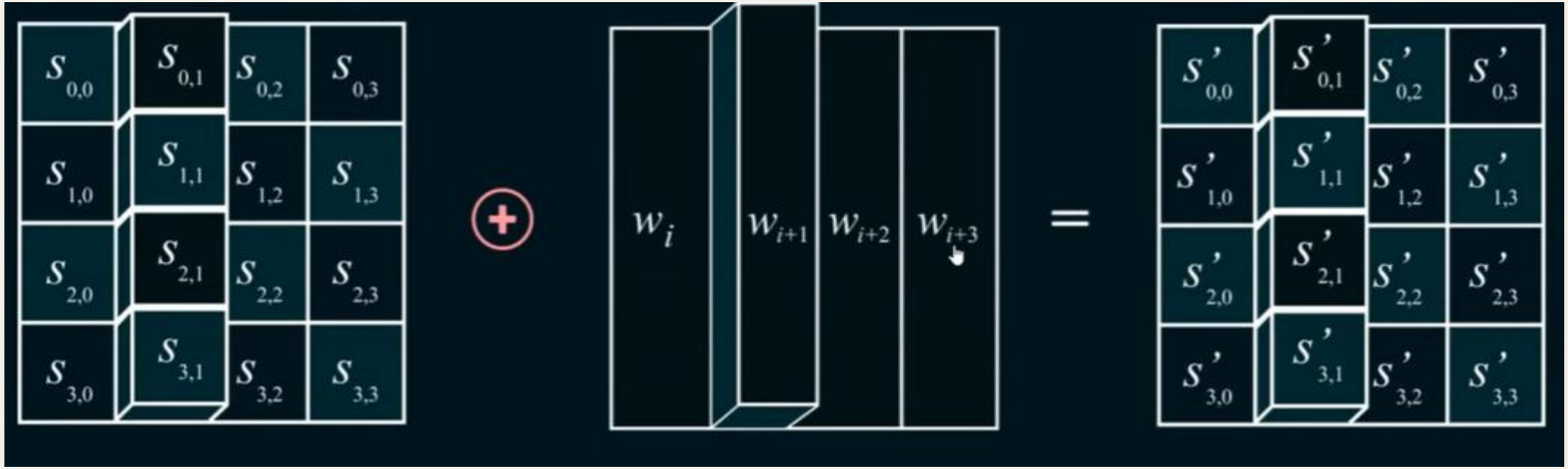
AddRoundKey

AES – AddRoundKey

- The ***AddRoundKey*** operates at the column level using **matrix addition** using **exclusive-OR (XOR)** operation.



AddRound Key. "i" is the round No.



- Round Key 0 = w0,w1,w2,w3
- Round Key 1 = w4,w5,w6,w7
- …
- Round Key 4 = w16,w17,w18,w19
- Encryption (StateNew = StateOld XOR RoundKey)
- Decryption (StateOld = StateNew XOR RoundKey)

AES Decryption

- AES Encryption

  SubBytes Substitution => ShiftRows => MixColumns => Add Round Key

- AES Decryption (not the same order)

  Inverse ShiftRows => Inverse SubBytes => Add RoundKey => MixColumn

- This is done for performance & Ease of implementation purposes

ADVANCED ENCRYPTION STANDARD (AES)

Key Expansion

AES – Key Expansion (Overview)

- AES uses a key-expansion process to **create round key** for each round.
- If the number of rounds is N_r (based on the AES version used), the key-expansion routine creates $(N_r + 1)$ of 128-bit round keys from one single 128-bit cipher key.
- The additional one key is used for the pre-round transformation ***AddRoundKey***.
- The key expansion routine creates round keys **word by word** (an array of **4 bytes**).
- In other words, AES-128 (10 rounds) has a total of 44 words, AES-192 has a total of 48 words, and AES-256 has a total of 52 words.

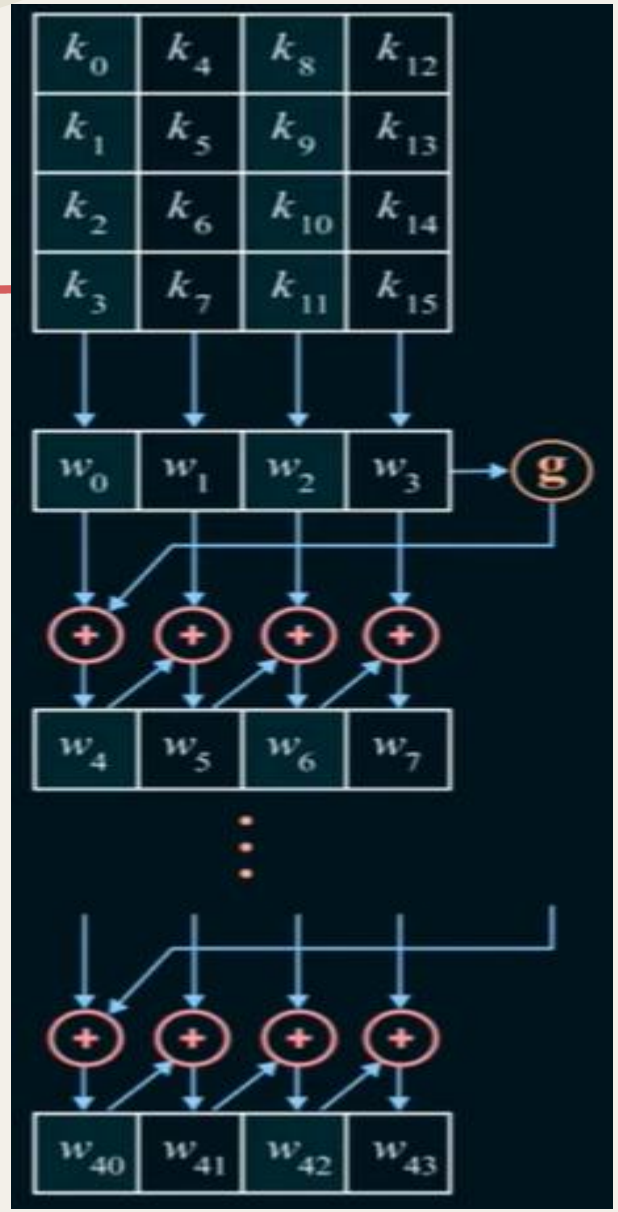
AES – Key Expansion (Overview)

- The following table shows the general words created for each round.

Round	*Words*			
Pre-round	$\mathbf{w}_0$	$\mathbf{w}_1$	$\mathbf{w}_2$	$\mathbf{w}_3$
1	$\mathbf{w}_4$	$\mathbf{w}_5$	$\mathbf{w}_6$	$\mathbf{w}_7$
2	$\mathbf{w}_8$	$\mathbf{w}_9$	$\mathbf{w}_{10}$	$\mathbf{w}_{11}$
. . .	. . .			
N_r	$\mathbf{w}_{4N_r}$	$\mathbf{w}_{4N_r+1}$	$\mathbf{w}_{4N_r+2}$	$\mathbf{w}_{4N_r+3}$

- As a demonstration, in next few slides, we consider the key expansion in **AES-128** by observing how the round key is created from words.

Key Expansion Overview



- Initial 128 bits key is put into 4 word w0,… w3 for round 0
- w4 = g(w3) ; g is a function
- w4 to w7 will feed as round 1 key
- w40 to w43 will feed as round 10 key
- If $(i \bmod 4) \neq 0$, $w_i = w_{i-1} \oplus w_{i-4}$ (for all "i" not multiple of 4)
- $(i \bmod 4) = 0$, $w_i = t \oplus w_{i-4}$ (the g function)
-

AES – Key Expansion (AES-128)

1. The first four words (w_0, w_1, w_2, w_3) are made from the **cipher key**. The cipher key is thought of as an array of 16 bytes $(k_0 \text{ to } k_{15})$. The first four bytes $(k_0 \text{ to } k_3)$ become w_0; the next four bytes $(k_4 \text{ to } k_7)$ become w_1; and so on.
2. The rest of the words (w_i for $i = 4, \dots, 43$) are made as follows:
   a. If $(i \bmod 4) \neq 0$, $w_i = w_{i-1} \oplus w_{i-4}$.
   b. If $(i \bmod 4) = 0$, $w_i = t \oplus w_{i-4}$. Here t, a temporary word, is the result of applying two routines, ***SubWord*** and ***RotWord***, on w_{i-1} and XOR the result with a **round constants**, ***RCon***. In other words, we have

$$t = \text{SubWord}\big(\text{RotWord}(w_{i-1})\big) \oplus \text{RCon}_{\frac{i}{4}}$$

AES – Key Expansion (AES-128)

- The **temporary word *t*** computations:

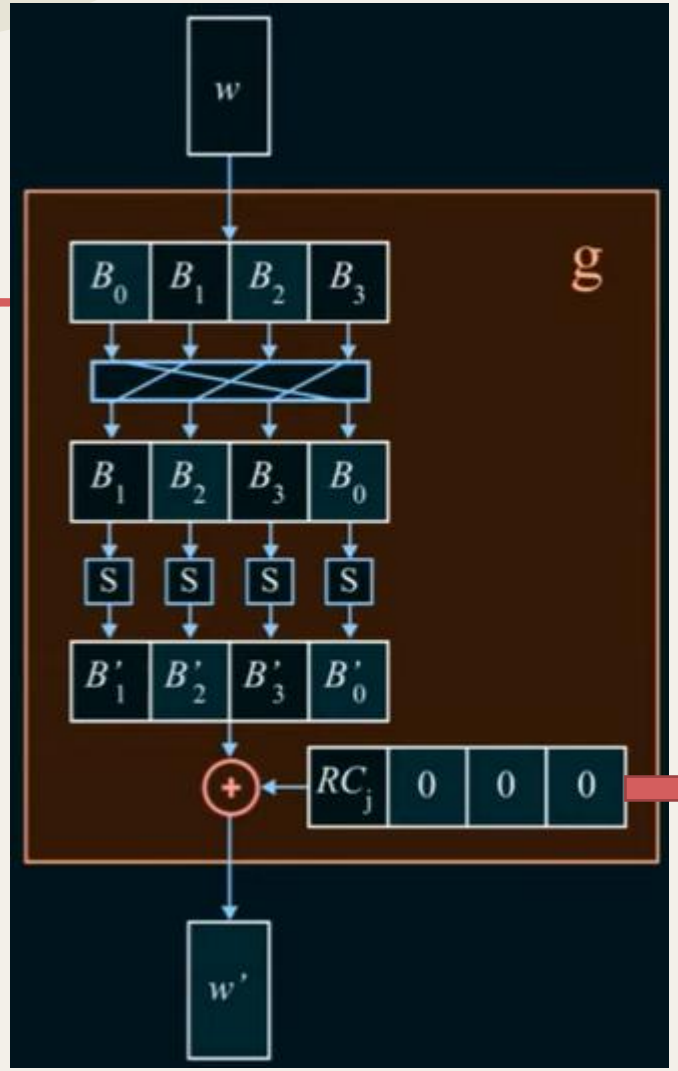
- ***1st: RotWord***: Similar to ***ShiftRows*** transformation but left-shift only 1 word to the left with wrapping.

- ***2nd: SubWord*** : Similar to ***SubBytes*** transformation.
  The exact same ***SubBytes*** table is used.

- ***3rd: XOR with Round Constant: RCon*** : A 4-byte value that rightmost 3 bytes are always zero.

Round	*Constant (RCon)*	*Round*	*Constant (RCon)*
1	$(\underline{\mathbf{01}}\ 00\ 00\ 00)_{16}$	6	$(\underline{\mathbf{20}}\ 00\ 00\ 00)_{16}$
2	$(\underline{\mathbf{02}}\ 00\ 00\ 00)_{16}$	7	$(\underline{\mathbf{40}}\ 00\ 00\ 00)_{16}$
3	$(\underline{\mathbf{04}}\ 00\ 00\ 00)_{16}$	8	$(\underline{\mathbf{80}}\ 00\ 00\ 00)_{16}$
4	$(\underline{\mathbf{08}}\ 00\ 00\ 00)_{16}$	9	$(\underline{\mathbf{1B}}\ 00\ 00\ 00)_{16}$
5	$(\underline{\mathbf{10}}\ 00\ 00\ 00)_{16}$	10	$(\underline{\mathbf{36}}\ 00\ 00\ 00)_{16}$

G-Function



Right 3 bytes all zeros

Inside the function

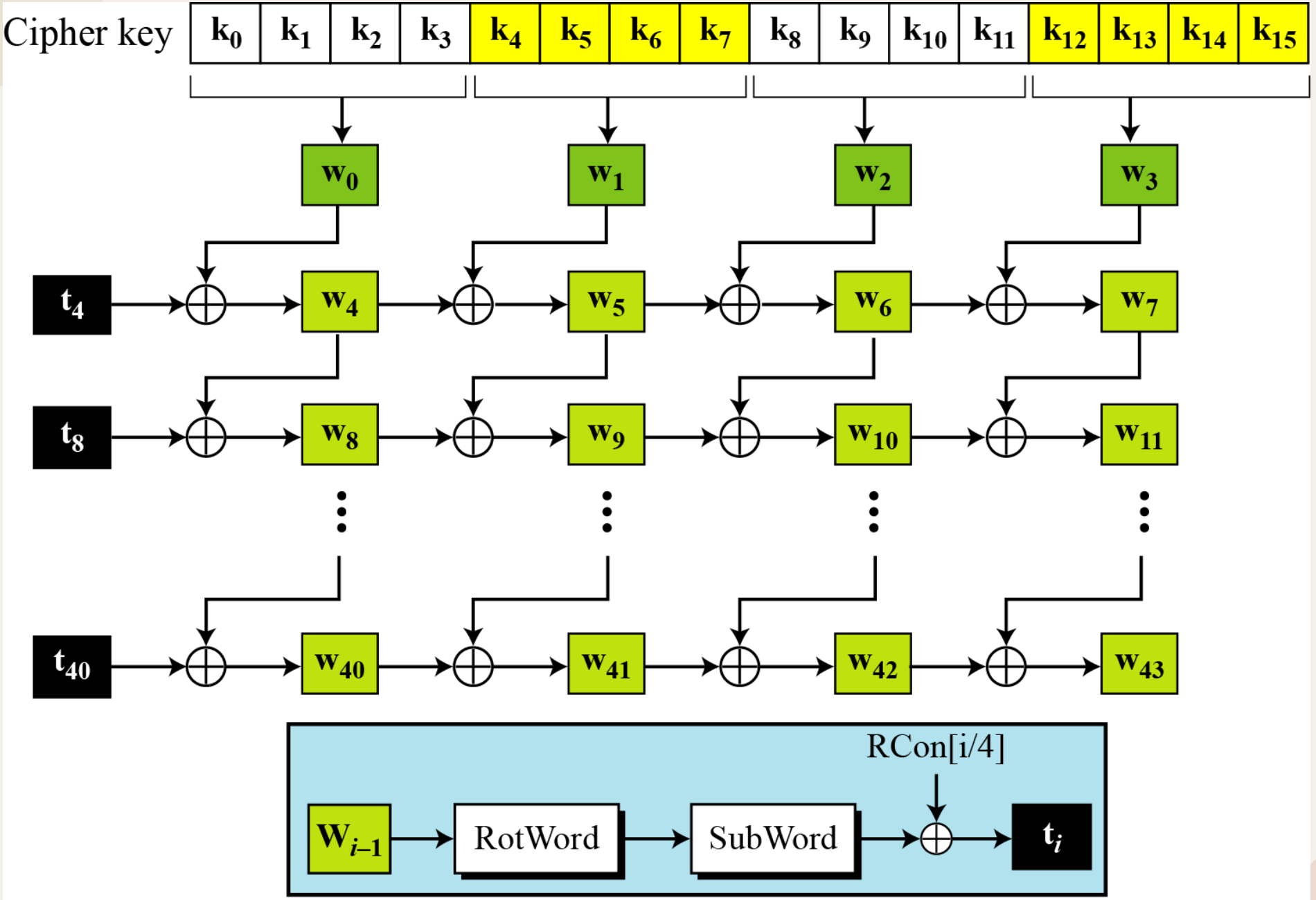
1st Rotate left 1 byte

2nd: SubBytes Transformation using the same SubBytes table

3rd: XOR with Rcon (refer to table below)

Note: g-function is a name only

Round	*Constant (RCon)*	*Round*	*Constant (RCon)*
1	$(\underline{\mathbf{01}}\ 00\ 00\ 00)_{16}$	6	$(\underline{\mathbf{20}}\ 00\ 00\ 00)_{16}$
2	$(\underline{\mathbf{02}}\ 00\ 00\ 00)_{16}$	7	$(\underline{\mathbf{40}}\ 00\ 00\ 00)_{16}$
3	$(\underline{\mathbf{04}}\ 00\ 00\ 00)_{16}$	8	$(\underline{\mathbf{80}}\ 00\ 00\ 00)_{16}$
4	$(\underline{\mathbf{08}}\ 00\ 00\ 00)_{16}$	9	$(\underline{\mathbf{1B}}\ 00\ 00\ 00)_{16}$
5	$(\underline{\mathbf{10}}\ 00\ 00\ 00)_{16}$	10	$(\underline{\mathbf{36}}\ 00\ 00\ 00)_{16}$



Making of t_i (temporary) words $i = 4\ N_r$

Algorithm 7.5 *Pseudocode for key expansion in AES-128*

```
KeyExpansion ([key0 to key15], [w0 to w43])
{
    for (i = 0 to 3)
        wi ← key4i + key4i+1 + key4i+2 + key4i+3

    for (i = 4 to 43)
    {
        if (i mod 4 ≠ 0)    wi ← wi–1 + wi–4
        else
        {
            t ← SubWord (RotWord (wi–1)) ⊕ RConi/4        // t is a temporary word
            wi ← t + wi–4
        }
    }
}
```

AES – Key Expansion (AES-128)

- **Example**: Suppose we want to compute the first temporary word t. Assuming that we have word $w = (13AA5487)$.

1. $\text{RotWord}(w) = \text{RotWord}(13AA5487) = AA548713$ (rotate left 1 byte)
2. $\text{SubWord}\big(\text{RotWord}(w)\big) = \text{SubWord}(AA548713) = AC20177D$ (refer next slides)
3. $t = \text{SubWord}\big(\text{RotWord}(w_0)\big) \oplus \text{RCon}_1 = AC20177D \oplus 01000000_{16} = AD20177D$ (assume round 1).

- Hence, the first temporary word $\boldsymbol{t = AD20177D}$. We will learn more about these computations in the tutorial sessions.

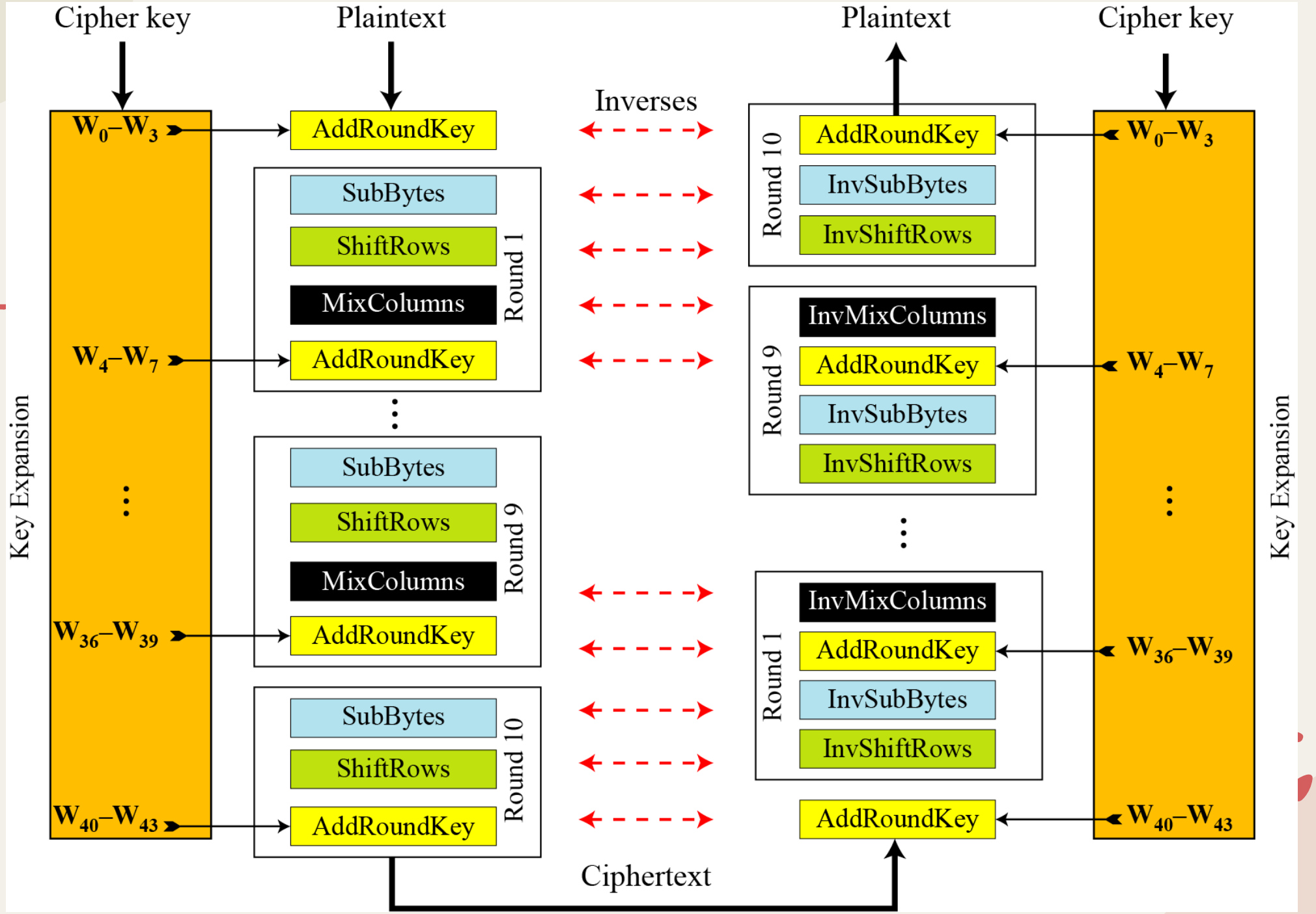
SubWord($AA548713$) = $AC20177D$

x \ y	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D 4	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20 2	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17 3	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC 1	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

(a) S-box

ADVANCED ENCRYPTION STANDARD (AES)

Complete AES Ciphers


Cipher key
Plaintext
Plaintext
Cipher key
Inverses
Key Expansion
Key Expansion
W0–W3
W4–W7
W36–W39
W40–W43
AddRoundKey
SubBytes
ShiftRows
MixColumns
AddRoundKey
Round 1
SubBytes
ShiftRows
MixColumns
AddRoundKey
Round 9
SubBytes
ShiftRows
AddRoundKey
Round 10
Round 10
InvSubBytes
InvShiftRows
Round 9
InvMixColumns
AddRoundKey
InvSubBytes
InvShiftRows
Round 1
InvMixColumns
AddRoundKey
InvSubBytes
InvShiftRows
AddRoundKey
Ciphertext

ADVANCED ENCRYPTION STANDARD (AES)

AES vs DES Comparative Analysis

AES – Non-Feistel Ciphers

- AES operates on the **ENTIRE block of 128-bit**, different from DES which split into 2 blocks of equal length and operates differently (left block and right block have different encryption process).
- The operations in AES (***SubBytes***, ***ShiftRows***, ***MixColumns***, ***AddRoundKeys***) are **INVERTIBLE** under Galois Field $GF(2^8)$, different from DES which the substitution (S-Box) are randomized and is **non-invertible**.
- AES is **iterative** cipher rather than Feistel cipher.

Bit & Bytes Orientations (Substitution)

Operation	AES	DES
	Byte-Oriented	Bit-Oriented
Substitution	▪ SubBytes Transformation takes a byte and change it to another byte. ▪ Need **only one** substitution table.	▪ S-Box takes 6-bit and transforms it to 4-bit. ▪ Need **8 S-boxes** to create a 32-bit half block.

Bit & Bytes Orientations (Permutation)

Operation	AES	DES
	Byte-Oriented	Bit-Oriented
Permutation	▪ ShiftRows transformation transposes **bytes**. ▪ Permutation is applied **straight on the bytes**. ▪ There is **no expansion** permutation needed.	▪ P-Box transposes **bits**. ▪ Permutation is applied **before and after substitution**. ▪ **Expansion** permutation is needed to create a 48-bit half block from a 32-bit half block. This is needed since the round key is 48-bit long.

AES versus DES – Properties

Property	AES	DES
Block Size	128-bit	64-bit
Round Key Size	128-bit	48-bit
Number of Rounds	10, 12, or 14	16
Cipher Key Size	128-bit, 192-bit, 256-bit	56-bit
Type of Cipher	**Non-Feistel** Cipher (Iterative) ▪ Operates on entire data block in every round.	**Feistel** Cipher ▪ Operates on halves of the data block at a time

ADVANCED ENCRYPTION STANDARD (AES)

AES Security

AES – Security

- All AES versions remain secure, and none has been broken so far.
- It survives against **brute-force** (longer key-length), **statistical attack** (confusion and diffusion), **differential and linear attack** (not applicable).
- Although AES-256 is the current **strong**, **practical**, and **recommended** symmetric-key algorithm, AES-128 is still widely used in my applications.
- The **strength** of AES-128 is **equivalent** to 3072-bit RSA.

Main References for Chapter 5

1. Stallings, W. "***Cryptography and Network Security: Principles and Practices***", Fifth Edition, Pearson Prentice Hall, 2011.
2. Forouzan, B.A. "***Introduction to Cryptography and Network Security***", Third Edition, McGraw-Hill, 2008.

~ THE END ☺ ~